

@bookepp2019discrete, author = Epp, Susanna S., title = Discrete Mathematics with Applications, edition = 5th, publisher = Cengage Learning, year = 2019

RSA Security Lab

Abigail Volante

July 10, 2026

Contents

1	Introduction	1
2	Prerequisites	2
3	RSA- How does it work?	2
4	Helpful Functions	2
4.1	Modular Binary Exponentiation Algorithm	3
4.2	Correctness of the Modular Binary Exponentiation Algorithm . .	4

1 Introduction

This document contains all the necessary explanations and proofs behind this project. As much as the mathematician in me would like this to be fully rigorous, I think it's more useful to strike a balance between rigour and understanding. My aim is to explain my thought process about the project, as well as how I actually wrote the code.

Most RSA implementations I've seen are presented without any argument that the code actually computes what the mathematics claims, and without examining what happens when implementation choices depart from the ideal. I aim to close this gap and to actually fully explain the theory behind RSA.

I do not attempt to give full proofs for every algorithm used here but only for the things where the proof is the actual point. This includes:

- RSA correctness
- `mod_pow`'s loop invariant
- Miller-Rabin's probabilistic error bound
- Why each attack works mathematically

For everything else, I give a logical explanation instead. Full proofs for those standard results are easy to find elsewhere.

2 Prerequisites

Knowledge of modular arithmetic and elementary proof techniques is recommended.

Throughout this project, I will be referring to an important theorem called *The Loop Invariant Theorem*. Its proof can be readily found in many resources.

Defintion 2.1. *The **guard** of a while loop is the condition restricting entry to the loop. The assertion just preceding the loop is called the **pre-condition for the loop**, and the one following is called the **post-condition for the loop**.*

Defintion 2.2. *A loop is **correct with respect to its pre- and post-conditions** if, and only if, whenever the algorithm variables satisfy the pre-conditions for the loop and the loop terminates after a finite number of steps, the arlgorithm variables satisfy the post-condition for the loop.*

Theorem 2.1 (Loop Invariant Theorem). *Let a while loop with a guard G be given, together with pre- and post-conditions that are predicates in the algorithm variables. Also, let a predicate $I(n)$ called the **loop invariant** be given. If the following four properties hold, then the loop is correct with respect to its pre- and post-conditions.*

1. **Basis Property:** *The pre-condition for the loop implies that $I(0)$ is true before the first iteration of the loop*
2. **Inductive Property** *For every integer $k \geq 0$, if the guard G and the loop invariant $I(k)$ are both true before an iteration of the loop, then $I(k + 1)$ is true after an iteration of the loop*
3. **Eventual Falsity of Guard** *After a finite number of iterations of the loop, the guard G becomes false.*
4. **Correctness of the Post-Condition** *If N is the least number of iterations after which G is false and $I(N)$ is true, then the values of the algorithm variables will be as specified in the post-condition of the loop.*

3 RSA- How does it work?

4 Helpful Functions

The very first thing we need to do is to write out helpful functions which we will be calling later when implementing the RSA code. This includes `mod_pow`, `extended_gcd`, and `mod_inverse`.

4.1 Modular Binary Exponentiation Algorithm

The `mod_pow` (modular power) function takes in values b, e , and n and outputs $b^e \bmod n$. We will be using this function when creating the encryption and decryption keys as well as for the Miller-Rabin test. Here is the exact algorithm we used in this project:

Modular Binary Exponentiation Algorithm

```
def mod_pow(base, exp, mod):
    result = 1
    base = base % mod
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        exp //= 2
        base = (base * base) % mod
    return result
```

How exactly do we calculate $b^e \bmod n$? For a demonstration, let us consider $3^{13} \bmod 5$. We first calculate $3 \bmod 5$ then using that result, calculate $3^2 \bmod 5$ then $3^4 \bmod 5$ and so on. We use the property:

$$ab \bmod n \equiv (a \bmod n)(b \bmod n) \quad \text{for any } a, b, n \in \mathbb{Z} \quad (1)$$

We find that

$$\begin{aligned} 3 &\equiv 3 \bmod 5 \\ 3^2 &\equiv 3^2 \equiv 9 \equiv 4 \bmod 5 \\ 3^4 &\equiv 3^2 3^2 \equiv 4 \cdot 4 \equiv 16 \equiv 1 \bmod 5 \\ 3^8 &\equiv 3^4 3^4 \equiv 1 \cdot 1 \equiv 1 \bmod 5. \end{aligned}$$

Writing $13 = 8 + 4 + 1$, we have

$$3^{13} \equiv 3^8 3^4 3^1 \equiv 1 \cdot 1 \cdot 3 \equiv 3 \bmod 5.$$

Notice how we didn't have to calculate $3^e \bmod 5$ until we get to $e = 13$. Instead, we only calculated the square powers of 3. This is actually, a more efficient way to do this. Calculating for every e until we get to 13 would require us to perform 13 multiplications whereas with this method, we only need 4. We call this method *Binary Exponentiation* and is the basis of this algorithm. In general, this algorithm has time complexity $O(\log e)$.

Notice that when we wrote $13 = 8 + 4 + 1$, we unintentionally have represented 13 in its binary form 1101 since:

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1$$

Essentially, writing a number e in its binary representation is to recursively perform the following. (The reader can easily check this holds.)

1. Is e divisible by 2? If yes, then e has a 1 in its last digit in its binary representation otherwise it has 0.
2. Replace e to be the its quotient on division by 2 and repeat.

This is exactly the structure of the while loop in `mod_pow`: `exp % 2` checks the current bit, and `exp //= 2` performs the halving step above. Whenever that bit is 1, `result` is multiplied by the current `base`; squaring `base` each iteration moves it from b^{2^k} to $b^{2^{k+1}}$, matching bit k of e 's binary expansion. The one modification is that every value is reduced modulo n throughout, keeping all numbers bounded by n even when e is large.

4.2 Correctness of the Modular Binary Exponentiation Algorithm

References

- [1] Epp, Susanna S. *Discrete Mathematics with Applications*, 5th ed. Cengage Learning, 2019.